HN Crawler — Distributed Crawler Project Specification (Tier 1)

Target: news.ycombinator.com (single domain). Server-rendered HTML, no JavaScript required. Four distinct entity types with genuine FK relationships — stories, comments, users, external domains. Enough volume (millions of stories, 30M+ comments) to justify distributed crawling. Rate-limits on sustained aggressive requests (503 above ~1 req/sec) — forces real retry/backoff logic.

HN's robots.txt allows crawling. ROBOTSTXT_OBEY = True. Crawl politely.

---

1. Spider Architecture (Mod 1)

HN Page Types

Four distinct page types, each requires separate callback:

List pages — `/, /news, /newest, /ask, /show, /jobs, /news?p=2` — render 30 story items per page. Contain: story HN id, title, external URL (or null for Ask/Show), domain, score, comment count, author username, age string. Pagination via "More" anchor at page bottom linking to `?p=N`. Max ~50 pages per section before HN truncates.

Story/comment pages — `/item?id=<hn_id>` — render story metadata repeated + all top-level and nested comments in single HTML response. No comment pagination — full thread on one page. Comments rendered as nested `<tr>` elements with indent level encoded in `width` attribute of spacer `<td>`. Depth derivable from spacer width ÷ 40.

User profile pages — `/user?id=<username>` — render: username, karma, account created date, about (bio, raw HTML with links). Single page, no pagination.

Submission link pages — external URLs linked from stories. Not crawled — external_url stored as text only, domain extracted and FK-linked to domains table.

Spider Type Decision

BaseSpider, not CrawlSpider — three reasons specific to HN:

1. Pagination is conditional: "More" link absent on final page. CrawlSpider's Rule evaluates link patterns before callback executes — cannot conditionally stop based on page state. BaseSpider callback controls whether to `response.follow(more_link)` or stop.
2. Single URL pattern (`/item?id=N`) covers both story pages and comment-only pages — no regex distinguishes them structurally. CrawlSpider Rule cannot dispatch

different callbacks based on page content. BaseSpider inspects response body to determine entity type and dispatches accordingly.

3. Three entity types from one domain require per-entity rate budgets and crawl priority logic. BaseSpider + manual `response.follow()` with per-request `meta` allows injecting `entity_type`, `priority`, and `crawl_budget` into each request independently.

Spider Architecture: Three Spiders, One Project

StoriesSpider — crawls list pages. Yields story items. Discovers story page URLs and user profile URLs, pushes to Redis frontier with entity_type meta. Does not follow story pages itself.

ItemSpider — crawls `/item?id=N` pages. Yields one story metadata update item + N comment items per page. Discovers commenter usernames, pushes user profile URLs to Redis frontier.

UserSpider — crawls `/user?id=<username>` pages. Yields user items.

Three spiders share one Redis instance, separate key namespaces: `hn_stories:requests`, `hn_items:requests`, `hn_users:requests`. Separate dupefilters. Independent CONCURRENT_REQUESTS settings per spider type.

Why separate spiders over one: different crawl rates per entity type (stories: 0.5 req/sec, items: 0.3 req/sec for deeper pages, users: 1 req/sec). Different retry strategies. Different pipeline stages active per type. CrawlSpider cannot do this cleanly.

start_requests Override

Override `start_requests()`, not `start_urls`. Inject into `Request.meta`:

- `crawl_run_id` — UUID, set once at spider open, never overwritten downstream
- `entity_type` — `'story_list'`, `'story_item'`, `'user'`
- `section` — `'front'`, `'newest'`, `'ask'`, `'show'`, `'jobs'` (from list pages only)
- `url_fingerprint` — set by RequestFingerprintMiddleware, not spider

Meta Propagation Contract

Every Request carries `crawl_run_id` and `entity_type` in meta from seed through all child requests. Callbacks read from `response.meta`, never re-derive. `entity_type` determines which item class is yielded and which pipeline stages activate.

ItemLoader Configuration

Three Item classes: `StoryItem`, `CommentItem`, `UserItem`. Separate ItemLoader per class.

Default processors (all loaders):

- Input: `MapCompose(str.strip)`
- Output: `TakeFirst()`

StoryItem field processors:

- `hn_id` input: `MapCompose(str.strip, int)` — HN id is integer, extracted from href or data attribute
- `score` input: `MapCompose(str.strip, remove_non_digits, int)` — "342 points" → 342
- `num_comments` input: `MapCompose(str.strip, remove_non_digits, int)` — "147 comments" → 147
- `external_url` input: `MapCompose(str.strip)` — null for Ask HN (no href on title)
- `domain` input: `MapCompose(str.strip, extract_domain)` — "github.com" from sitestr span, not parsed from URL (HN renders it separately)
- `created_at` input: `MapCompose(parse_hn_age)` — HN renders "3 hours ago", "2 days ago" — convert to UTC timestamp using crawl time as reference anchor

CommentItem field processors:

- `hn_id` input: `MapCompose(int)` — from comment anchor id attribute
- `depth` input: `MapCompose(extract_indent_width, lambda w: w // 40)` — spacer td width ÷ 40
- `text` output: `Join(separator=' ')` — comment text spans multiple elements
- `parent_id` input: `MapCompose(int)` — from reply link href `?parent=N`

UserItem field processors:

- `karma` input: `MapCompose(str.strip, int)`
- `created_at` input: `MapCompose(parse_hn_date)` — HN renders exact date for users ("November 14, 2012") unlike stories

URL Canonicalization

HN tracking params to strip before SHA1 fingerprint: none (HN uses clean URLs). Still apply `w3lib.url.canonicalize_url` for: lowercase scheme+host, remove default port, sort query params. Query param sort matters: `/news?p=2` must canonicalize identically regardless of future param additions. Standard strip list (`utm_*`, `fbclid`, `gclid`, `ref`)

applied defensively even though HN doesn't use them — dupefilter and middleware must use same fn.

---

2. Item Pipeline + Middleware Stack (Mod 2)

Item Pipeline Order

Stage 1 — Priority 100: ValidationPipeline Required fields by entity type:

- StoryItem: `hn_id`, `title`, `author`, `created_at`, `crawled_at`, `url_fingerprint`
- CommentItem: `hn_id`, `story_id`, `crawled_at`, `url_fingerprint`
- UserItem: `username`, `crawled_at`, `url_fingerprint`

On missing required field: `raise DropItem`, log entity_type + field name + page URL. Optional field absence (score=null on job posts, text=null on deleted comments): log warning, pass item through with null.

Stage 2 — Priority 200: EntityDedupPipeline Dedup logic differs per entity_type:

- Stories: query `stories` table by `hn_id`. If exists AND `last_crawled` within 6 hours: drop (score/comment count unlikely changed). If stale: upsert score + num_comments + last_crawled. If absent: insert.
- Comments: query `comments` by `hn_id`. If exists: drop (comments immutable after ~2hr edit window). If absent: insert.
- Users: query `users` by `username`. If exists AND `last_crawled` within 24 hours: drop. Else: upsert karma + about.

HN comments are effectively immutable after edit window — no content-hash dedup needed, existence check sufficient.

Stage 3 — Priority 300: ContentHashDedupPipeline Active only for StoryItem and UserItem (comment immutability handled in Stage 2).

Story content hash: `SHA256(str(score) + "|" + str(num_comments))` — detects score/comment count change on re-crawl. If hash matches last stored hash for this hn_id: drop (no change). Hash must use normalized string representations (no trailing spaces, consistent null handling).

User content hash: `SHA256(str(karma) + "|" + STRIP(about or ''))` — detects karma change.

Stage 4 — Priority 400: PostgresPipeline Single DB connection per spider instance. Batch size 50. Flush on batch full or `close_spider()`. Per entity_type: routes to correct INSERT

statement with correct ON CONFLICT clause. If DB unavailable: write items to local JSONL fallback file named `fallback_<crawl_run_id>_<timestamp>.jsonl`. Log critical. Do not drop data silently.

Stage 5 — Priority 500: URLDiscoveryPipeline Active only for StoryItem and CommentItem. On new story found: push `/item?id=<hn_id>` URL to `hn_items:start_urls` Redis List (for ItemSpider). On new author/commenter found: push `/user?id=<username>` to `hn_users:start_urls` Redis List (for UserSpider). Dedup via Redis SADD to `hn_items:url_discovery_seen` and `hn_users:url_discovery_seen` sets before push — prevents redundant pushes across pipeline runs.

Stage 6 — Priority 600: StatsPipeline Increment: `stories_stored`, `stories_dropped_dup`, `comments_stored`, `comments_dropped_dup`, `users_stored`, `users_dropped_dup`, `items_dropped_validation`. Written to `crawl_runs` on spider close.

Middleware Stack

Middleware 1 — Priority 100: CrawlIdInjectMiddleware `process_request`: inject `crawl_run_id` UUID from `self.crawl_run_id` (set in `spider_opened` signal) if absent in meta. Never overwrite — preserves ID through retries and redirects.

Middleware 2 — Priority 200: RequestFingerprintMiddleware `process_request`: `canonicalize_url(url)` → strip tracking params → `SHA1` hex. Store as `request.meta['url_fingerprint']`. Identical computation to ItemLoader's `url_fingerprint` field. Divergence between middleware and loader = fingerprint mismatch between dupefilter entry and stored URL record.

Middleware 3 — Priority 300: RandomUserAgentMiddleware `process_request`: select UA from pool. Pool stratified: 60% Chrome/Windows, 20% Chrome/macOS, 15% Firefox/Windows, 5% Safari/macOS. Mirrors real browser distribution — uniform random is statistically detectable. Store selected profile key in `request.meta['ua_profile']`.

Middleware 4 — Priority 350: ConsistentHeaderMiddleware `process_request`: read `request.meta['ua_profile']`. Inject matching header set:

- Chrome/Win: `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8` | `Accept-Language: en-US,en;q=0.9` | `Accept-Encoding: gzip, deflate, br` | `Sec-Fetch-Dest: document` | `Sec-Fetch-Mode: navigate` | `Sec-Fetch-Site: none` | `Sec-Fetch-User: ?1`
- Firefox/Win: different Accept-Language priority order, absent Sec-Fetch-User

- Safari/macOS: different Accept value, no Sec-Fetch-* headers (Safari omits them entirely)

Mismatch between UA string and Sec-Fetch headers is a primary TLS fingerprinting signal. Correct per-profile injection eliminates this.

Middleware 5 — Priority 400: ProxyRotationMiddleware `process_request`: select proxy. Sticky per domain (HN is single domain — one sticky proxy per crawl session is fine). On 503: rotate proxy + UA, add current proxy to TTL blacklist (300 seconds). On all proxies blacklisted: pause requests for 60 seconds via `spider.crawler.engine.pause()`, resume after TTL expires. `process_response`: on 407: rotate proxy, return copied request.

Note: HN is single domain. Proxy rotation on HN is primarily for IP-based rate limit evasion, not cross-domain session management.

Middleware 6 — Priority 500: SoftBlockDetectorMiddleware `process_response`: inspect every response regardless of status. HN-specific soft-block signals: response body length < 200 bytes (HN minimum valid page is ~2KB), response URL changed to `/blocked` or `/login` (redirect to login = session invalidation signal), body contains literal string `"Sorry, we're not able to serve your requests"` (HN's rate limit page text, status 200). On detect: inc `soft_block_count` stat, rotate proxy+UA, return `request.copy()`.

Middleware 7 — Priority 550: RetryMiddleware (override) `RETRY_HTTP_CODES`: 429, 500, 502, 503, 504, 408. HN returns 503 (not 429) on rate limit — 503 must be in retry codes. Do NOT retry 404 (HN deletes stories, 404 is real). Do NOT retry 301/302 — follow redirects normally.

Backoff: `min(2 * 2^attempt, 60) * uniform(0.8, 1.2)`. Jitter prevents synchronized retry storms from distributed workers. On exhaustion: insert `failed_requests` record directly via DB, log `retry_exhausted_count`.

---

3. Anti-Bot Layer (Mod 4)

Extension of Middleware 3–6. HN-specific rate limit behavior documented here.

HN Rate Limit Behavior

HN returns HTTP 503 (not 429) when request rate exceeds ~1 req/sec sustained to the same endpoint class. Endpoint classes: list pages, item pages, user pages — each has independent rate budget server-side. Response body at 503: `"Sorry, we're not able to serve your requests this fast. Please slow down and we'll get you back up to speed. More information."` — exact string, use for soft-block detection.

Rate Control Settings

`AUTOTHROTTLE_ENABLED = True` — adaptive throttle. HN response latency varies: fast during off-peak, slow during US business hours when load spikes. Fixed DOWNLOAD_DELAY cannot adapt; AUTOTHROTTLE responds to latency signals.

`AUTOTHROTTLE_TARGET_CONCURRENCY = 1` — one concurrent request to HN at a time per worker. HN is single domain; multiple concurrent requests from same IP triggers rate limit faster than high per-request rate.

`AUTOTHROTTLE_START_DELAY = 2.0` — conservative initial delay before calibration.

`AUTOTHROTTLE_MAX_DELAY = 60.0` — allow up to 60s delay if HN is slow. Do not time out on slow response.

`CONCURRENT_REQUESTS_PER_DOMAIN = 1` — hard cap. With 2 workers: max 2 effective concurrent requests to HN. Acceptable for interview-scale crawl.

`ROBOTSTXT_OBEY = True` — HN robots.txt allows all paths. Set True anyway: demonstrates compliance awareness.

`DOWNLOAD_TIMEOUT = 30` — HN pages load in < 2s under normal conditions. 30s timeout catches genuine hangs without dropping slow legitimate responses.

Proxy Infrastructure (Describe, Not Build)

Proxy tiers for interview:

- Residential proxies: real ISP IPs, highest trust level, slowest, most expensive. Use for targets with aggressive IP fingerprinting.
- Datacenter proxies: fast, cheap, easily detected/blocked by IP range analysis. Sufficient for HN (HN does not block datacenter ranges).
- ISP proxies: datacenter IPs registered to ISP ASNs. Hybrid cost/trust.

For HN specifically: datacenter proxies sufficient. HN rate limits by request rate, not by IP reputation. Rotation strategy: sticky per crawl session (one proxy per spider worker instance), rotate only on 503 or soft-block detect.

---

4. PostgreSQL Schema (Mod 8)

Design Rationale

HN's data is genuinely relational with 4 entity types:

- `stories` — core entity, references `users` (author) and `domains` (external link domain)
- `comments` — references `stories` (story_id) and self (parent_id) and `users` (author)
- `users` — referenced by both `stories` and `comments`
- `domains` — referenced by `stories`

HN uses its own integer IDs for stories and comments. These are the natural primary keys — use HN's IDs directly (BIGINT), not BIGSERIAL. Avoids dual-ID complexity. HN IDs are globally unique monotonically increasing integers across all item types.

Table: domains

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
domain	TEXT	NOT NULL, UNIQUE
first_seen	TIMESTAMPTZ	NOT NULL, DEFAULT now()
story_count	INT	DEFAULT 0

`domain` = registered domain extracted from story external_url (e.g. `github.com`, not `gist.github.com` — strip subdomain or keep full, pick one convention and document it). `story_count` updated via trigger or pipeline batch on insert — used for domain frequency queries without GROUP BY at query time.

Table: users

Column	Type	Constraints
username	TEXT	PRIMARY KEY
karma	INT	nullable
created_at	TIMESTAMPTZ	nullable

Column	Type	Constraints
about	TEXT	nullable
first_crawled	TIMESTAMPTZ	NOT NULL, DEFAULT now()
last_crawled	TIMESTAMPTZ	nullable

`username` is natural PK — HN usernames are unique, immutable, never reassigned. No surrogate key needed. Avoids FK join through surrogate id everywhere `users` is referenced — TEXT FK is fine at this cardinality.

`karma` nullable: UserSpider may not have crawled user yet when story/comment first seen. Story pipeline inserts username into users table with null karma on first encounter; UserSpider fills it later.

Table: stories

Column	Type	Constraints
hn_id	BIGINT	PRIMARY KEY
title	TEXT	NOT NULL
external_url	TEXT	nullable
domain_id	INT	nullable, FK → domains(id)
score	INT	DEFAULT 0
num_comments	INT	DEFAULT 0
author	TEXT	NOT NULL, FK → users(username)
story_type	TEXT	NOT NULL, CHECK IN ('story', 'ask', 'show', 'job')
created_at	TIMESTAMPTZ	NOT NULL
first_crawled	TIMESTAMPTZ	NOT NULL, DEFAULT now()
last_crawled	TIMESTAMPTZ	nullable
content_hash	CHAR(64)	nullable

`hn_id` = HN's own integer. PRIMARY KEY.

`external_url` null for Ask HN and Show HN posts where title is the content. `domain_id` null when `external_url` null.

`story_type` derived from title prefix ("Ask HN:", "Show HN:") and presence of external URL.

`content_hash` = SHA256(str(score) + "|" + str(num_comments)) stored per row. Compare on re-crawl to detect change. Avoids ContentHashDedupPipeline DB query — pipeline computes hash, compares against this column via `SELECT content_hash FROM stories WHERE hn_id = X`.

Table: comments

Column	Type	Constraints
hn_id	BIGINT	PRIMARY KEY
story_id	BIGINT	NOT NULL, FK → stories(hn_id)
parent_id	BIGINT	nullable
author	TEXT	nullable, FK → users(username)
text	TEXT	nullable
depth	SMALLINT	NOT NULL, DEFAULT 0
created_at	TIMESTAMPTZ	nullable
first_crawled	TIMESTAMPTZ	NOT NULL, DEFAULT now()

`parent_id` nullable: null = top-level comment (direct reply to story). Non-null = reply to another comment. Note: parent_id is not a FK to comments table — parent may not yet be crawled or may be a deleted comment. Enforcing FK here causes insert ordering dependency. Store as bare BIGINT, application-level join.

`author` nullable: deleted comments have no author. `text` nullable: deleted comments render as `[deleted]` — store null, not literal string.

`depth` derived from spacer `<td>` width attribute divided by 40. Depth 0 = top-level. Required for thread reconstruction query without recursive CTE.

Table: crawl_runs

Column	Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
spider_name	TEXT	NOT NULL
started_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
finished_at	TIMESTAMPTZ	nullable
status	TEXT	CHECK IN ('running', 'completed', 'failed', 'stopped')
settings_snapshot	JSONB	nullable
items_scraped	INT	DEFAULT 0
items_failed	INT	DEFAULT 0
soft_blocks	INT	DEFAULT 0
bytes_downloaded	BIGINT	DEFAULT 0

`settings_snapshot` JSONB: key settings at crawl start — CONCURRENT_REQUESTS, AUTOTHROTTLE_TARGET_CONCURRENCY, RETRY_TIMES. Enables reproducing prior run's behavior.

Table: urls

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
url	TEXT	NOT NULL
url_fingerprint	CHAR(40)	NOT NULL, UNIQUE
entity_type	TEXT	NOT NULL, CHECK IN ('story_list', 'story_item', 'user')
first_seen	TIMESTAMPTZ	NOT NULL, DEFAULT now()
last_crawled	TIMESTAMPTZ	nullable
http_status	SMALLINT	nullable

Column	Type	Constraints
depth	SMALLINT	nullable
is_seed	BOOLEAN	DEFAULT FALSE

`entity_type` added vs generic schema — HN has single domain, so domain_id FK is redundant. entity_type replaces it as the primary classification dimension for this single-domain crawl. Enables per-entity crawl scheduling queries.

Table: crawl_results (partitioned)

Column	Type	Constraints
id	BIGSERIAL	— (composite PK with crawled_at)
crawl_run_id	UUID	NOT NULL, FK → crawl_runs(id)
url_id	BIGINT	NOT NULL, FK → urls(id)
entity_type	TEXT	NOT NULL
hn_id	BIGINT	nullable
content_hash	CHAR(64)	NOT NULL
content_ref	TEXT	nullable
crawled_at	TIMESTAMPTZ	NOT NULL
PRIMARY KEY	(id, crawled_at)	— partition key required in PK

PARTITION BY RANGE (crawled_at), monthly. Partition naming: `crawl_results_2026_01`. Pre-create 2 months ahead minimum.

`content_ref` = path to gzipped raw HTML: `crawl/<crawl_run_id>/<url_fingerprint>.html.gz`. Raw HTML off-row — item pages (stories + full comment threads) can be 200KB+.

`hn_id` allows JOIN from crawl_results to stories/comments without going through urls table. Nullable because list pages don't map to a single hn_id.

Table: failed_requests

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
crawl_run_id	UUID	FK → crawl_runs(id)
url	TEXT	NOT NULL
url_fingerprint	CHAR(40)	nullable
entity_type	TEXT	nullable
fail_reason	TEXT	NOT NULL
last_attempt	TIMESTAMPTZ	NOT NULL
attempt_count	SMALLINT	NOT NULL

`fail_reason` controlled vocabulary: `RETRY_EXHAUSTED`, `SOFT_BLOCK`, `HTTP_503`, `HTTP_4XX`, `TIMEOUT`, `VALIDATION_FAIL`. GROUP BY fail_reason gives health breakdown per run.

---

5. SQL Indexing & Query Tuning (Mod 9)

Index Definitions

stories table:

- `INDEX ON stories (created_at DESC)` — btree. Time-series queries: "stories in last 24h", "stories this week." Most common filter.
- `INDEX ON stories (score DESC, created_at DESC)` — composite btree. "Top stories by score in last N days." score DESC as leading column for pure ranking queries; crawled_at in WHERE prunes by time. Wait - `created_at DESC` is better as leading sort column when there's a range filter on it. Actually for "top stories this week" you'd filter on created_at first, then sort by score. But for "all-time top stories" you sort by score alone. Composite `(score DESC, created_at DESC)` serves both: equality/range on score, sort on created_at. Keep as written.
- `INDEX ON stories (domain_id, score DESC)` — composite btree. "Top stories from github.com." domain_id equality first (high selectivity), score sort second.

- `INDEX ON stories (author, created_at DESC)` — composite btree. "Stories posted by user X, newest first."
- `PARTIAL INDEX ON stories (hn_id) WHERE last_crawled IS NULL` — stories discovered via list page (hn_id known) but item page not yet crawled.
- `INDEX ON stories (story_type, score DESC)` — "top Ask HN posts of all time", "top Show HN posts."

comments table:

- `INDEX ON comments (story_id, depth ASC, created_at ASC)` — composite btree. Thread reconstruction: all comments for story, ordered by depth then time. story_id equality first.
- `INDEX ON comments (author, created_at DESC)` — commenter activity queries.
- `PARTIAL INDEX ON comments (story_id, hn_id) WHERE parent_id IS NULL` — top-level comments only. Smaller than full index. Used for "how many top-level threads on story X."
- `INDEX ON comments (created_at DESC)` — time-series comment queries, recent activity.

users table:

- `INDEX ON users (karma DESC)` — leaderboard query: "top 100 users by karma."
- `PARTIAL INDEX ON users (username) WHERE last_crawled IS NULL` — users discovered but never crawled.
- `INDEX ON users (created_at ASC)` — "oldest accounts" query, account age analysis.

urls table:

- `UNIQUE INDEX ON urls (url_fingerprint)` — btree. Point lookup on every dedup check. Most frequent query.
- `INDEX ON urls (entity_type, last_crawled DESC NULLS FIRST)` — composite btree. "All story_item URLs not crawled recently." entity_type equality first, last_crawled sort second. NULLS FIRST = never-crawled URLs surface first.
- `PARTIAL INDEX ON urls (id) WHERE http_status IS NULL` — never-fetched URLs only. Crawl queue building.

crawl_results partitions:

- `UNIQUE INDEX ON crawl_results (url_id, content_hash)` — dedup across runs. ON CONFLICT clause in pipeline uses this.

- `INDEX ON crawl_results (crawled_at DESC)` — partition pruning enabler. Without crawled_at in WHERE clause, all partitions scanned.
- `INDEX ON crawl_results (url_id, crawled_at DESC)` — "latest result for this URL."
- `INDEX ON crawl_results (entity_type, crawled_at DESC)` — "all comment pages crawled today."

failed_requests table:

- `INDEX ON failed_requests (crawl_run_id, last_attempt DESC)` — per-run failure report.
- `INDEX ON failed_requests (url_fingerprint)` — cross-run failure history for URL.

Composite Index Column Order

Leading column = equality filter first (highest cardinality per query). Trailing = range or sort. Violation example: `(created_at, story_type)` cannot serve `story_type = 'ask' AND created_at > X` efficiently — Postgres cannot use B-tree range on leading column then equality on trailing. Correct: `(story_type, created_at DESC)`.

Benchmark Queries

Query 1: "Top 50 stories from domain github.com in last 7 days by score."

- Filter: `domain_id = (SELECT id FROM domains WHERE domain = 'github.com')` + `created_at > NOW() - INTERVAL '7 days'`
- Sort: `ORDER BY score DESC LIMIT 50`
- Expected plan: Index Scan on `(domain_id, score DESC)` index, filter on created_at inline. No Seq Scan.

Query 2: "Full comment thread for story 12345678, ordered for display."

- Filter: `story_id = 12345678`
- Sort: `ORDER BY depth ASC, created_at ASC`
- Expected plan: Index Scan on `(story_id, depth ASC, created_at ASC)`. No Sort node in plan if index covers order.

Query 3: "Users discovered in last 24h not yet crawled."

- Filter on urls: `entity_type = 'user' AND first_seen > NOW() - INTERVAL '24h' AND http_status IS NULL`

- Expected plan: hits `PARTIAL INDEX WHERE http_status IS NULL` + entity_type + first_seen filter.

Query 4: "Comment count per top-level commenter this week."

- `SELECT author, COUNT(*) FROM comments WHERE created_at > NOW() - INTERVAL '7 days' AND parent_id IS NULL GROUP BY author ORDER BY COUNT(*) DESC LIMIT 20`
- Expected plan: Partial index scan on `WHERE parent_id IS NULL`, filter on created_at, hash aggregate. Verify `(author, created_at DESC)` index used vs seq scan on partial index result.

Run all four with `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)`. Document before/after each index creation. `shared_hit` high = buffer cache, `read` high = disk — first run after VACUUM will show disk read, subsequent runs buffer hit.

---

6. Distributed Crawling — Scrapy-Redis (Mod 6)

Redis Key Namespace (Three Spiders)

Key	Type	Spider	Purpose
`hn_stories:requests`	ZSET	StoriesSpider	Scheduler queue
`hn_stories:dupefilter`	SET	StoriesSpider	URL fingerprints
`hn_stories:start_urls`	LIST	StoriesSpider	Seed injection
`hn_items:requests`	ZSET	ItemSpider	Scheduler queue
`hn_items:dupefilter`	SET	ItemSpider	URL fingerprints
`hn_items:start_urls`	LIST	ItemSpider	Seed injection
`hn_users:requests`	ZSET	UserSpider	Scheduler queue
`hn_users:dupefilter`	SET	UserSpider	URL fingerprints
`hn_users:start_urls`	LIST	UserSpider	Seed injection
`hn_items:url_discovery_seen`	SET	URLDiscoveryPipeline	Prevents duplicate pushes

Key	Type	Spider	Purpose
`hn_users:url_discovery_seen`	SET	URLDiscoveryPipeline	Prevents duplicate pushes

ZSET = Sorted Set, score = request priority. SET = unordered set for O(1) membership test. LIST = queue via LPUSH/BRPOP.

Scheduler Queue: ZSET Mechanics

`scrapy_redis.queue.PriorityQueue` — Redis ZSET. ZADD with score = priority integer. ZREVPOPMIN pops highest score. Default priority 0 for all discovered URLs. StoriesSpider seeds `/newest` with higher priority (score=10) than `/news?p=5` (score=1) — ensures fresh stories discovered before deep-pagination pages.

ItemSpider priority scoring: story pages with high `num_comments` (known from list page meta) get higher ZSET score — more comments = more items per page = higher throughput value. Inject `num_comments` into Request meta from StoriesSpider → URLDiscoveryPipeline reads it when pushing to `hn_items:start_urls`, uses it as score in `ZADD hn_items:requests`.

Dupefilter: SET Mechanics

`scrapy_redis.dupefilter.RFPDupeFilter` — SADD fingerprint to SET. Returns 1 (new) → process request. Returns 0 (exists) → drop request. Atomic — no race between two workers checking then inserting same fingerprint.

Override `request_fingerprint()` in RFPDupeFilter subclass with same canonicalize+strip function as RequestFingerprintMiddleware. If these diverge: URL passes dupefilter using raw-URL SHA1, gets different fingerprint from middleware's canonical-URL SHA1 — two fingerprints for same URL in DB = constraint violation or missed dedup.

Scrapy-Redis Settings Per Spider

```
SCHEDULER = "scrapy_redis.scheduler.Scheduler"
DUPEFILTER_CLASS = "scrapy_redis.dupefilter.RFPDupeFilter"  # subclassed
SCHEDULER_PERSIST = True          # queue survives restart
SCHEDULER_QUEUE_CLASS = "scrapy_redis.queue.PriorityQueue"
CLOSESPIDER_TIMEOUT not set       # workers stay alive waiting for seeds
REDIS_URL = "redis://:password@host:6379/0"
REDIS_START_URLS_KEY = "{name}:start_urls"  # name = spider.name
```

Per-spider CONCURRENT_REQUESTS:

- StoriesSpider: 2 (list pages, moderate rate)
- ItemSpider: 1 per worker (item pages are large, 1 concurrent is sufficient per worker)
- UserSpider: 2 per worker (user pages are small)

Seed Injection Pattern

Seed injection decoupled from spider code. External cron script runs every N minutes:

1. Push list page URLs to `hn_stories:start_urls` via LPUSH
2. Push `/newest` with priority score 10 (freshest stories)
3. Push `/news`, `/ask`, `/show` with priority score 5
4. Push `/news?p=2` through `/news?p=5` with priority score 1

Spiders do not restart to pick up new seeds — they block on Redis queue, consume seeds as pushed. Worker count decision: monitor `hn_items:requests` ZSET size. Growing backlog = add ItemSpider workers. Empty backlog = StoriesSpider not discovering fast enough.

In-Flight Request Loss

Default scrapy-redis: ZREVPOPMIN removes from ZSET atomically. If worker crashes post-pop, pre-store: URL lost forever from frontier.

Mitigation (describe in interview): Two-ZSET reliable queue pattern:

- `hn_items:requests` — main queue
- `hn_items:processing` — in-flight tracking

Pop from main: `MULTI / ZPOPMIN hn_items:requests / ZADD hn_items:processing score member / EXEC` (atomic transaction). On item successfully stored in Postgres: `ZREM hn_items:processing member`. Watchdog process: scan `hn_items:processing` for members with score (timestamp) older than 10 minutes, re-push to main queue via `ZADD`. This is the Redis reliable queue pattern.

Implementation complexity is high — describe the pattern, note it as production consideration vs study project simplification.

Tracking Param Fingerprint Divergence

HN URLs are clean (no tracking params). However, scraped comment href attributes occasionally include `?goto=item%3Fid%3D...` redirect URLs. Custom strip list applied: `goto`, `utm_*`, `ref`, `source`. Same list in both RequestFingerprintMiddleware and RFPDupeFilter subclass. Divergence = split-brain dedup state across workers.

Dedup Layer Summary

Layer	Mechanism	Where	Cost	Catches
1	Redis SADD to dupefilter SET	Pre-HTTP request	Cheapest, O(1)	Exact URL dups across workers in same run
2	Postgres UNIQUE url_fingerprint on urls table	Pipeline insert	Cheap, indexed lookup	Exact URL dups across runs
3	Postgres UNIQUE (url_id, content_hash) on crawl_results	Pipeline insert	Medium	Same URL, same content, different run
4	content_hash column on stories/users tables	Pipeline query	Medium	Entity-level change detection per re-crawl

Layer 4 is HN-specific addition to generic spec — entity tables store content_hash directly, enabling single-row lookup for change detection instead of separate crawl_results query.

Monitoring Surface

Metrics stored in `crawl_runs`, updated on `spider_closed` signal:

- `items_scraped` — from Scrapy built-in `item_scraped_count` stat
- `items_failed` — from StatsPipeline custom counter
- `soft_blocks` — from SoftBlockDetectorMiddleware custom `soft_block_count`
- `bytes_downloaded` — from `downloader/response_bytes`
- `elapsed_seconds` — `finished_at - started_at`

Derived health metrics (compute via SQL on crawl_runs):

- Throughput: `items_scraped / elapsed_seconds`
- Soft-block rate: `soft_blocks / (items_scraped + soft_blocks)`
- Failure rate: `items_failed / (items_scraped + items_failed)`

Alert thresholds:

- Soft-block rate > 5% → rate too aggressive or IP reputation problem
- Failure rate > 10% → HN HTML structure change (selector broke) or systematic block

- Throughput < 0.5 items/sec per worker → autothrottle over-backing-off, investigate latency

Per-entity breakdown query: `SELECT entity_type, COUNT(*) FROM crawl_results WHERE crawl_run_id = X GROUP BY entity_type` — verifies all three spiders producing output.